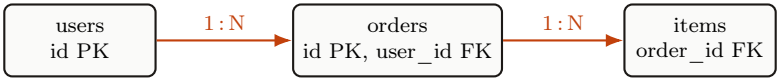
SQL Databases Review

Analysis and Assessment of Design Quality

Relational model in one frame



Tables + keys + constraints
Declarative query language (SQL)
Engine enforces schema & integrity

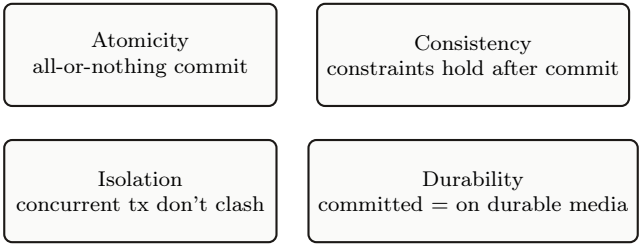
Shared schema, joins, and transactional writes — not “any store that speaks SQL dialect.”

Client → engine → storage



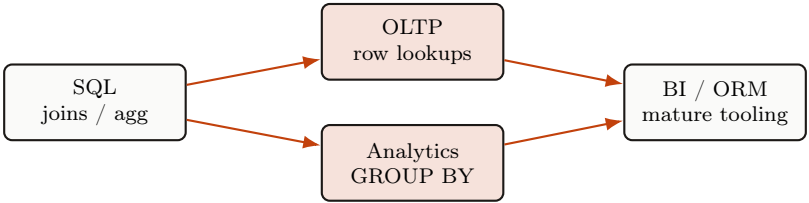
one logical database
strong consistency by default

ACID — the contract (names only)



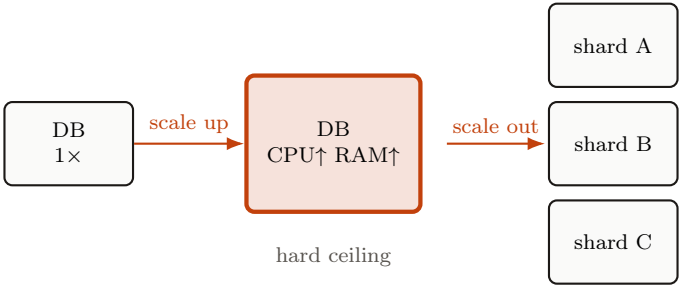
Review lens: which business invariants need this contract vs eventual OK?

Ad-hoc querying & analytics



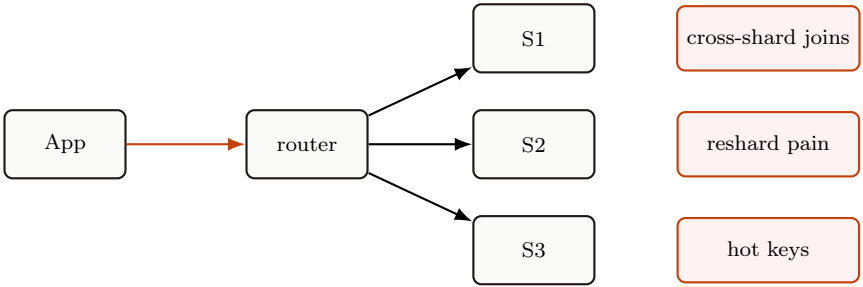
New question $\neq$ new service — schema + indexes often enough.

Vertical ceiling → horizontal pressure



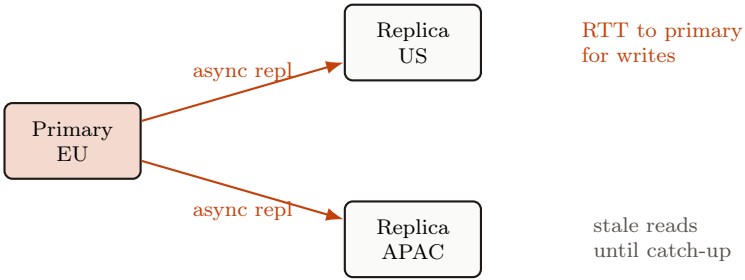
One primary writer eventually saturates — then you partition data, not just compute.

Sharding — power and tax



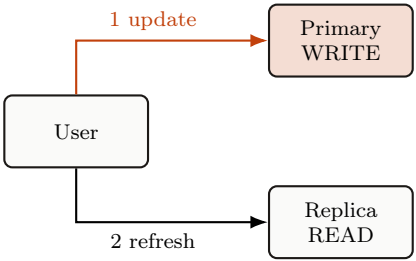
Capacity grows; transactional simplicity and ops cost often do not.

Latency + replica lag



Multi-region SQL is possible — consistency vs latency is the real design choice.

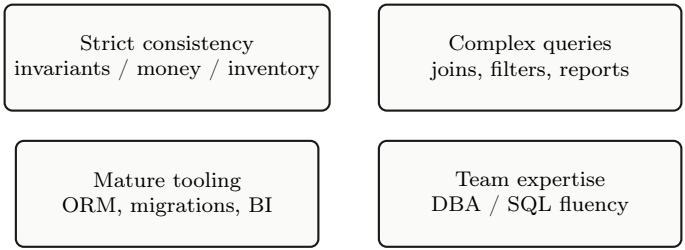
Write here, read there — stale?



may miss
own write

Mitigations: read-your-writes sticky, sync replica, or accept lag for that path.

Choose SQL when . . .



Default store for most transactional domains until a limit is proven.

Quiz — fit

Quiz

Billing ledger needs serialisable balance updates and ad-hoc finance joins. Best default?

1. Key-value store only — simpler ops always wins
2. Relational primary with ACID; scale tactics before abandoning SQL
3. Search index as system of record
4. Sharded wide-column first, migrate to SQL later

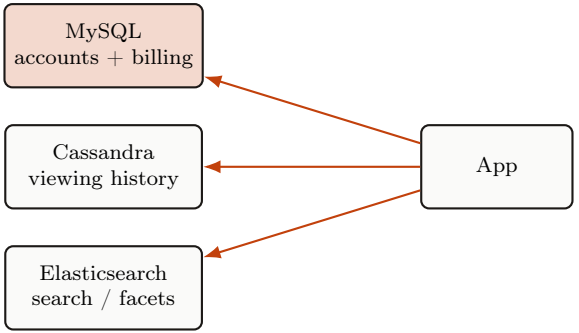
Pause — discuss with a neighbour or answer privately before the next slide.

Answer — fit

Answer

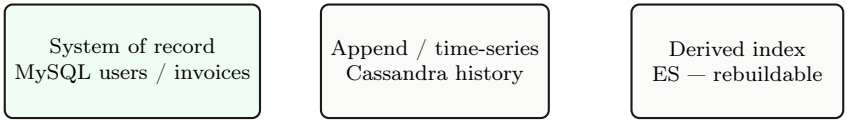
2. Consistency + complex joins are SQL's home ground; prove a limit before a wholesale rewrite.

One DB ≠ every access pattern



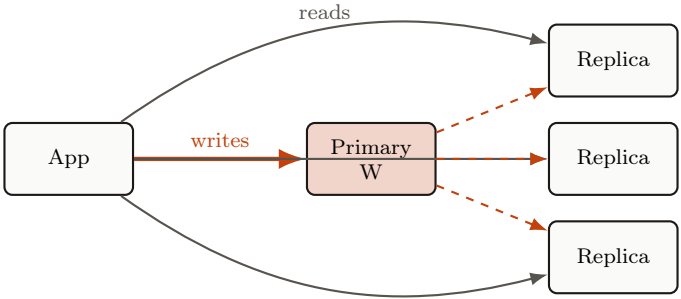
Polyglot = intentional boundaries — not “every shiny DB in the org chart.”

Ownership boundaries



Know which store is source of truth; indexes and caches must be rebuildable.

Read replicas — write primary



First lever for read-heavy workloads — still one write bottleneck.

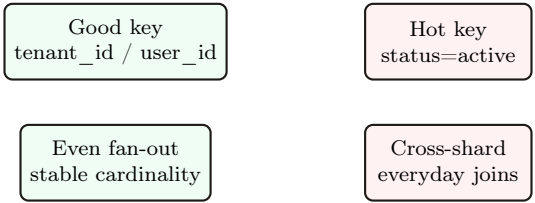
Pool + cache in front of SQL



illustrative: GitHub-style caching
cut DB queries ∼90% on hot paths

Connection pooling caps open sessions; cache absorbs repeated reads.

Sharding key — choose carefully



Key choice locks query shape and reshard cost for years.

Quiz — scaling tactic

Quiz

Read:write ≈ 40:1; single primary CPU high on SELECTs; writes fine. First move?

1. Immediate multi-region active-active
2. Read replicas + pooling; cache hot keys; defer shard until write wall
3. Drop ACID and move ledger to ES
4. One bigger VM only — forever

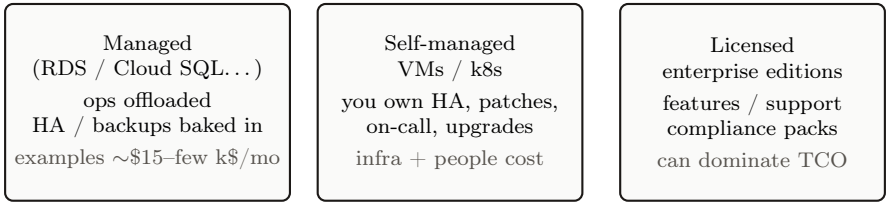
Pause — discuss with a neighbour or answer privately before the next slide.

Answer — scaling tactic

Answer

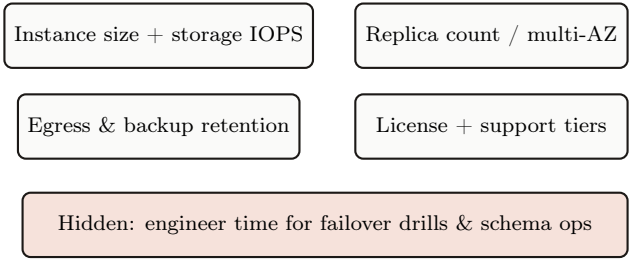
2. Read path first (replicas, pool, cache); sharding is for write/data volume walls.

Managed vs self-managed vs license



Ranges are illustrative examples only — quote your cloud for real numbers.

Cost drivers to review



SQL as default — and its edges

Best default
consistency + queries + tools

Scales far
often to millions of users

Hits walls
write primary, geo lag, shards

Architect takeaway
know limits + migration plan

Review snapshot

- ☐ ACID needed on this path?
- ☐ read vs write bottleneck?
- ☐ replica lag acceptable?
- ☐ polyglot SoR clear?
- ☐ pool / cache / shard key / exit plan?